



Лекція 4

Багатопоточність. Рефлексія. Аннотації

Багатопоточність є одним із ключових аспектів розробки програм на Java. Це підхід, який дозволяє програмі виконувати кілька операцій одночасно, забезпечуючи краще використання ресурсів комп'ютера та підвищити продуктивність програм.

Багатопоточність у Java була придумана, щоб вирішити два основних завдання:

1. Одночасно виконувати кілька дій.
2. Прискорити обчислення.



Процеси

Процес (Process): — це сукупність коду та даних, що поділяють загальний віртуальний адресний простір. Кожен процес має свою власну область пам'яті, в якій зберігається його власний код програми, дані, стек викликів, та інші ресурси. Кожен процес має власне середовище виконання та може включати один чи більше потоків.

Потік виконання (Thread)

Потік - одиниця виконання коду всередині процесу, що виконується послідовно.

Один **Процес** може запускати один або більше **Потоків**, що виконуються паралельно (або псевдопаралельно).

Кожний потік має свою область у пам'яті, що називається **Стеком**.

Стек складається з **Фреймів** (відповідають точкам виклику методів), в фреймах зберігаються параметри методу та локальні змінні.

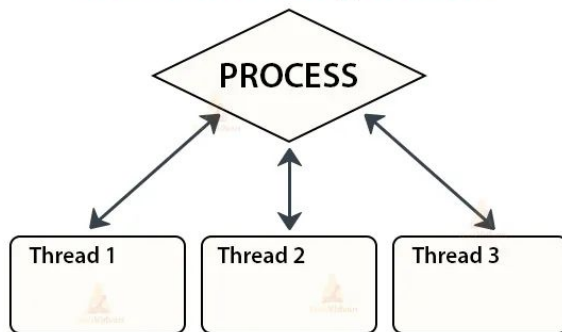
Детальніше тут <https://www.artima.com/insidejvm/ed2/jvm8.html>

Переваги багатопоточності

Переваги багатопоточності полягають у:

- підвищенні продуктивності роботи додатків;
- можливості програмувати довготривалі операції в окремих потоках, не розбиваючи їх на окремі відрізки.

Multithreading in Java



Створення потоку

Для створення потоку в Java слугує клас **Thread**.

Існує 2 варіанти створення потоку:

- шляхом наслідування класу **Thread**;
- шляхом реалізації інтерфейсу **Runnable**.

Створення потоку - наслідника Thread

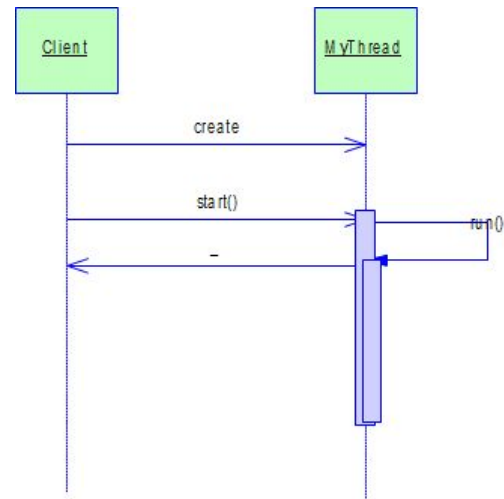
Найпростішим способом створення потоку є створення класу, що є похідним від **Thread**, з перевизначеним у ньому методом **run()**.

Далі слід створити об'єкт новоствореного класу та викликати його метод **start()**.

Метод **start()** виконує системні дії щодо створення потоку, після чого викликає перевизначений нами метод **run()**.

З цього моменту новий потік почне своє існування. Він буде виконуватися паралельно з іншими потоками, конкуруючи з ними за час процесору.

Коли відбудеться вихід з методу **run()**, потік завершить свою роботу.



Приклад №1 створення потоку

Кожну секунду потік виводить повідомлення на екран:

```
public class NewThread extends Thread {  
    @Override  
    public void run() {  
        try {  
            for (int i = 1; i <= 5; i++) {  
                System.out.println("Дочірній потік: " + i);  
                Thread.sleep(1000);  
            }  
        } catch (InterruptedException ex) {  
            // Виключення може викинути метод sleep()  
            System.out.println("Дочірній потік перерваний.");  
        }  
        System.out.println("Дочірній потік завершений.");  
    }  
}
```

З головного потоку Main створимо дочірній потік:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Головний потік починається...");  
        // Створюємо і запускаємо дочірній потік  
        NewThread newThread = new NewThread();  
        newThread.start();  
        // Головний потік продовжує виконання  
        System.out.println("Головний потік завершений.");  
    }  
}
```


Приклад №2 створення потоку

Ще простий приклад:

```
public class MyFirstThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("I'm Thread! My name is " + getName());  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 0; i < 10; i++) {  
            MyFirstThread thread = new MyFirstThread();  
            thread.start();  
        }  
    }  
}
```

Результат у консоль

Висновок у консоль:

I'm Thread! My name is Thread-2

I'm Thread! My name is Thread-1

I'm Thread! My name is Thread-0

I'm Thread! My name is Thread-3

. . .

Пояснення:

- Створюємо **10 потоків** класу `MyFirstThread`, що успадковує `Thread`.
- Метод `start()` запускає кожен потік, який виконує свій `run()`.
- **Порядок виконання непередбачуваний** — його визначає **планувальник потоків** операційної системи.

Зверніть увагу: імена потоків йдуть не по порядку. Справа в тому, що ми в даному випадку лише віддаємо команди на створення та запуск 10 потоків.

В якому порядку їх запускати – вирішує планувальник потоків: особливий механізм всередині операційної системи.

Створення потоку шляхом реалізації інтерфейсу Runnable

Цей спосіб використовується, коли наслідування від класу **Thread** неможливе (наприклад, клас вже наслідується від іншого класу).

Кроки, які необхідно здійснити:

- створити клас **MyThread**, що буде реалізовувати інтерфейс **Runnable**, та реалізувати в ньому метод **run()**;
- створити екземпляр цього класу;
- створити об'єкт **Thread**, передавши йому в конструктор екземпляр класу **MyThread**;
- викликати метод **start()** об'єкту **Thread**.

Приклад створення потоку

```
public class MyThread implements Runnable {  
    public void run() {  
        // тіло методу те ж,  
        // що і у попередньому прикладі  
    }  
}
```

З головного потоку створимо дочірній потік:

```
MyThread myThread = new MyThread();  
Thread thread = new Thread(myThread);  
thread.start();
```

1. **MyThread** - клас, який реалізує інтерфейс **Runnable**.
2. Створили новий об'єкт класу **Thread**, передали йому в конструкторі об'єкт **myThread**, чий метод **run()** потрібно буде виконати.
3. Запустили новий потік у роботу за допомогою виклику методу **start()**.

Диспетчеризація потоків

Паралельна робота потоків на одному процесорі забезпечується операційною системою.

Для того, щоб забезпечити правильну передачу керування між потоками на будь-якій ОС, потік повинен періодично віддавати управління операційній системі.

Для цих цілей в класі **Thread** слугують 2 методи:

- **sleep(long millis)** – призупиняє роботу потоку на задану кількість мілісекунд;
- **yield()** – не призупиняє роботу потоку, а просто дає можливість ОС виконати, за необхідності, переключення на інший потік або процес.

В будь-якому методі **run** за необхідності можна використовувати або **sleep**, або ж **yield**, або будь-який їх еквівалент (**wait** або методи читання з файлу). В протилежному випадку можна отримати ефект монопольного захоплення ресурсів машини однією ниткою.

Пріоритети потоків

Пріоритети потоків визначають порядок передачі управління між ними.

Правила передачі керування наступні:

- Потік з вищим пріоритетом отримує управління швидше.
- Якщо потік відмовився від управління (yield або sleep), його місце може зайняти потік з більшим пріоритетом.
- Якщо пріоритети рівні — рішення приймає операційна система.

Константи визначені в класі **Thread**.

За замовчуванням всі потоки мають стандартний пріоритет — **NORM_PRIORITY**.

Пріоритети потоків

Максимальний пріоритет (`MAX_PRIORITY`): Це найвищий рівень пріоритету, який може бути призначений потоку. Потоки з максимальним пріоритетом отримують перевагу при плануванні виконання та зазвичай отримують більше процесорного часу.

Стандартний пріоритет (`NORM_PRIORITY`): Це типовий рівень пріоритету, який надається потокам за замовчуванням, якщо інший рівень пріоритету не встановлено явно. Він зазвичай має середні значення та конкурує за ресурси з іншими потоками.

Мінімальний пріоритет (`MIN_PRIORITY`): Це найнижчий рівень пріоритету. Потоки з мінімальним пріоритетом можуть бути запущені рідше та мати менше часу виконання порівняно з потоками інших рівнів пріоритету.

Ми можемо змінювати пріоритет потоку за допомогою методу `setPriority()`. ОС буде враховувати ці значення під час планування роботи потоків — але не завжди гарантує точний порядок виконання. І головне — пріоритет не означає, що потік з найвищим рівнем **завжди** отримує процесор першим. Це лише підказка для планувальника, не абсолютне правило.

Поточний потік

Посилання на поточний об'єкт **Thread** можна отримати за допомогою статичного методу **Thread.currentThread()**.

```
public static void main(String[] args) {  
    // Виводить ім'я поточного потоку  
    System.out.println(Thread.currentThread().getName());  
    // main  
}
```

Всі статичні методи класу **Thread** відносяться саме до поточного потоку.

Наприклад, **Thread.sleep(1000);**

Імена потоків

Потоку можна присвоїти ім'я:

- Через конструктор `Thread(String name);`
- Через метод `setName(String name).`

Поточне значення ім'я потоку можна отримати за допомогою методу **`getName()`**.

Ім'я потоку не використовується виконуючою системою та слугує для зручності програміста.

Якщо ім'я не задано вручну — Java сама призначає стандартне:

- головний потік має ім'я `main`,
- інші автоматично отримують `Thread-0`, `Thread-1`, і так далі.

Потоки-демони

Потік-демон, як правило, надає певні загальні послуги та працює у фоновому режимі допоки працює програма.

Незавершені потоки-демони руйнуються автоматично після завершення останнього з потоків користувача.

Для забезпечення і перевірки такої властивості потоків слугують два методи класу **Thread**:

- **boolean isDaemon()** – перевірка чи є потік демоном;
- **setDaemon(boolean on)** - встановлює/знімає ознаку демону.

Потоки-демони (Приклад)

У цьому прикладі створюються два потоки: `userThread` і `daemonThread`. `userThread` - це звичайний потік користувача, а `daemonThread` - демон-потік.

Потік `daemonThread` встановлюється як демон за допомогою методу `setDaemon(true)` перед його запуском. Після запуску обидва потоки виводять повідомлення кожної секунди протягом 5 секунд. Оскільки `daemonThread` є демон-поток, програма завершується, коли завершується основний потік, або коли завершується основна програма.

```
public class DaemonThreadExample {  
  
    public static void main(String[] args) {  
        Thread userThread = new Thread(new WorkerThread());  
        Thread daemonThread = new Thread(new DaemonWorkerThread());  
  
        // Встановлюємо демон-потік  
        daemonThread.setDaemon(true);  
  
        userThread.start();  
        daemonThread.start();  
  
        System.out.println("Main thread finished.");  
    }  
}
```

Потоки-демоны (Приклад)

```
public class WorkerThread implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("User thread executing: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class DaemonWorkerThread implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 100; i++) {
            System.out.println("Daemon thread executing: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Стан перегонів (Race condition)

Стан перегонів виникає, коли один і той же ресурс використовується декількома потоками одночасно, і в залежності від порядку дій кожного потоку може бути декілька можливих результатів. ЧЧ

Для уникнення стану перегонів важливо коректно синхронізувати доступ до спільних ресурсів, використовуючи механізми синхронізації, такі як блокування (locks), атомарні операції або інші засоби, що гарантують правильну взаємодію між потоками. Така синхронізація дозволяє забезпечити правильну та безпечну роботу багатопоточних програм.

Стан перегонів (Race condition)

Одним з підходів до уникнення стану перегонів є використання атомарних операцій та засобів синхронізації.

Наприклад, в Java для маніпулювання цілими числами в багатопоточних програмах можна використовувати класи з пакету **java.util.concurrent.atomic**, що забезпечують атомарні операції.

Наприклад:

AtomicInteger, AtomicBoolean, AtomicLong

AtomicIntegerArray: Представляє масив цілих чисел, над яким можна виконувати атомарні операції.

Ключове слово **volatile**

Вирішує проблему видимості та робить зміну значення атомарною, через те, що має відношення **happens-before**: запис до **volatile**-змінної здійснюється до будь-якого послідуячого зчитування **volatile**-змінної.

Пакет **java.util.concurrent.atomic** містить набір класів, які підтримують складові атомарні дії над одним значенням без блокування, подібно до **volatile**.

Використовуючи класи **Atomic**, можна реалізувати атомарну операцію **check-then-act**.

І **AtomicInteger**, й **AtomicLong** мають атомарну операцію інкременту/декременту.

ThreadLocal

Один із способів зберігати дані в потоці та зробити блокування необов'язковим — це використовувати сховище **ThreadLocal**.

ThreadLocal - це клас в Java, який дозволяє створювати змінні, які будуть локальними для кожного потоку. Це означає, що кожний потік матиме свою власну копію змінної, і зміни в одному потоці не впливатимуть на значення змінної в інших потоках.

ThreadLocal гарантує лише те, що кожний потік отримає посилання на свій об'єкт, але не ізолює самі об'єкти. Якщо два різних потоки покладуть в **ThreadLocal** один і той самий об'єкт, то при доступі до нього будуть виникати всі проблеми багатопоточності.

ThreadLocal

У нашому прикладі ми створюємо **ThreadLocal** змінну `threadLocal`, і кожний потік отримує доступ до своєї власної копії цієї змінної.

Ось як працює наш код:

Головний потік встановлює значення "**MAIN**" для змінної `threadLocal`.

Потік `firstThread` і потік `secondThread` створюються з **MyRunnable** об'єктом як задачею для виконання.

Кожний потік виводить поточне значення `threadLocal` перед тим, як встановити нове значення.

Кожний потік встановлює власне значення для `threadLocal`.

Потік головного методу чекає завершення `firstThread` і `secondThread`.

Потім він виводить значення `threadLocal` в головному потоці, яке залишилося встановленим в "**MAIN**".

Оскільки кожний потік має свою власну копію змінної `threadLocal`, зміни, внесені одним потоком, не впливають на значення змінної в інших потоках. Це дозволяє створювати безпечні для багатопоточного використання змінні, які легко здійснюють доступ до кожного потоку окремо.

Проблема множинного доступу

- коли декілька потоків поділяють один ресурс, то буває, що необхідно синхронізувати їхню роботу таким чином, щоб тільки один потік мав доступ до ресурсу в кожний момент часу;
- доступ к ресурсу регулюється за допомогою монітору;
- **монітор** – це об'єкт, що використовується для взаємовиключного блокування потоків (**mutually exclusive lock**);
- коли один потік починає виконувати будь-який з методів об'єкту-монітору (кажуть, входить в монітор), всі інші потоки підходячи до монітору зупиняються та чекають доки монітор не звільниться;
- у Java монітором може слугувати будь-який об'єкт. Ця властивість успадковується від класу **Object**.

Засоби синхронізації потоків у Java

На рівні об'єкту можна синхронізувати:

Метод. При виклику такого методу блокується об'єкт, для якого викликаний даний метод:

```
public void synchronized f() {  
    ...  
}
```

Ділянка коду.

```
synchronized(ref) {    // ref – посилання на об'єкт, що блокується  
    ...  
}
```

Коли один з потоків входить до критичної ділянки (**synchronized** метод або блок), що пов'язаний з блокуванням якогось об'єкту, то інші потоки не можуть потрапити до всіх інших критичних ділянок, що пов'язані з блокуванням того ж об'єкту, допоки потік, що захопив об'єкт, не вийде з критичної ділянки.

Приклад синхронізації ділянки коду

```
public class MyObject {  
    // В даному класі оголошено 2 синхронізованих методи f() та g()  
  
    private String name;  
  
    public MyObject(String name) {  
        this.name = name;  
    }  
  
    public synchronized void f() {  
        System.out.println("f() started for object" + name);  
  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
  
        System.out.println("f() finished for object" + name);  
    }  
  
    public synchronized void g() {  
        System.out.println("g() started for object" + name);  
  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
  
        System.out.println("g() finished for object" + name);  
    }  
}
```

Приклад синхронізації ділянки коду

```
public static void main(String[] args) {  
    MyObject myObj = new MyObject("MyObject1");  
  
    // Виклик потоків з синхронізацією за MyObject1  
    Thread1 t1 = new Thread1(myObj);  
    t1.start();  
    Thread2 t2 = new Thread2(myObj);  
    t2.start();  
    Thread3 t3 = new Thread3(myObj);  
    t3.start();  
  
    // Виклик потоків з синхронізацією за MyObject2  
    MyObject myObj2 = new MyObject("MyObject2");  
    Thread1 t4 = new Thread1(myObj2);  
    t4.start();  
}
```

Приклад синхронізації ділянки коду

// Потік, в якому викликається метод f()

```
public class Thread1 extends Thread{  
    private MyObject myObj;  
    public Thread1(MyObject myObj){  
        this.myObj = myObj;  
    }  
    public void run(){  
        myObj.f();  
    }  
}
```

// Потік, в якому викликається метод g()

```
public class Thread2 extends Thread{  
    private MyObject myObj;  
    public Thread2(MyObject myObj){ this.myObj = myObj; }  
    public void run(){  
        myObj.g();  
    }  
}
```

// Зверніть увагу, що об'єкт MyObject передається даним потокам у якості параметру

Приклад синхронізації ділянки коду

В даному потоці оголошена критична секція, що пов'язана з об'єктом MyObject.

```
static class Thread3 extends Thread {  
    private MyObject myObj;  
    public Thread3(MyObject myObj) {this.myObj = myObj;}  
  
    public void run() {  
        synchronized (myObj) {  
            System.out.println("synchronized section started for object" + myObj.name);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
            System.out.println("synchronized section finished for object" + myObj.name);  
        }  
    }  
}
```

Приклад синхронізації ділянки коду

Результати запуску програми

f() started for object MyObject1

f() started for object MyObject2

f() finished for object MyObject1

g() started for object MyObject1

f() finished for object MyObject2

g() finished for object MyObject1

synchronized section started for object MyObject1

synchronized section finished for object MyObject1

Усі потоки, що працюють із **MyObject1**, виконувались **послідовно**, тому що вони блокують один і той самий об'єкт.

Потік, який працював із **MyObject2**, виконувався **паралельно**, бо це інший об'єкт із власним монітором

.

Синхронізація статичних ділянок

В якості синхронізуючого об'єкту може виступати сам клас.

В цьому випадку послідовно виконуються всі ділянки коду, що синхронізовані за цим класом.

```
static class MyClass {  
    public synchronized static void f() {  
        System.out.println("f() started");  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
        System.out.println("f() finished");  
    }  
}  
  
static class ThreadS extends Thread{  
    public void run(){  
        MyClass.f();  
    }  
}
```

Синхронізація статичних ділянок

```
public static void main(String[] args) {  
  
    ThreadS ts = new ThreadS();  
    ts.start();  
  
    synchronized(MyClass.class){  
        System.out.println("We are in synchronized section");  
    }  
}
```

Результат роботи:

f() started

f() finished

We are in synchronized section

Як бачимо, ділянка, що синхронізована за **MyClass.class** очікує допоки не виконається метод **f()**.

Методи `wait()`, `notify()`, `notifyAll()`

Ці методи застосовуються для забезпечення взаємодії між потоками при роботі з об'єктом.

Вони визначені в класі **Object**, що означає те, що можуть бути викликані для всіх об'єктів.

- **`wait()`** – блокує виконання потоку, з якого він викликаний, і одночасно розблоковує об'єкт, для якого він викликаний;
- **`notify()`** - пробуджує один із потоків, які викликали **`wait()`** на тому ж самому моніторі;
- **`notifyAll()`** – пробуджує всі потоки. Першим буде виконуватися потік із найвищим пріоритетом.

Ці методи слід викликати тільки у разі, якщо об'єкт слугує монітором, тобто із **синхронізованого** коду.

Взаємодія потоків на прикладі

Задача. Є виробник та споживач даних. Існує сховище, що здатне зберігати товари, приймати та видавати товар зі сховища. Поки виробник не виробив продукт, споживач не може його купити. Нехай виробник повинен виготовити 5 товарів, відповідно споживач має їх всі купити. Але при цьому одночасно на складі може бути не більше 3 товарів. Для вирішення цього завдання задіємо методи `wait()` і `notify()`:

Програма складається із 3-х класів:

- **Producer** - потік-виробник даних;
- **Consumer** - потік-споживач даних;
- **Store** - сховище.

Взаємодія потоків на прикладі

```
class Producer implements Runnable {  
    Store store; // Посилання на об'єкт-сховище  
  
    Producer(Store store) { // Конструктор потік-виробник  
        this.store = store;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 5; i++)  
            store.put(); // Виробник викликає метод щоб додати товар  
    }  
}
```

Взаємодія потоків на прикладі

```
class Consumer implements Runnable {  
    Store store; // Посилання на об'єкт-сховище  
  
    // Конструктор потік-споживач  
    Consumer(Store store) {  
        this.store = store;  
    }  
  
    // Споживач отримує товар зі сховища  
    public void run() {  
        for (int i = 1; i <= 5; i++)  
            store.get();  
    }  
}
```

Взаємодія потоків на прикладі

```
public class Store {  
    private int product = 0; // Кількість продуктів, що зберігається у сховищі  
    synchronized int get() {  
        while (product < 1) { // Якщо сховище пусте, потік, що прийшов за даними, повинен чекати  
            try {  
                wait(); // блокує виконання потоку, з якого він викликаний  
            } catch (p ex) {}  
        }  
        product--;  
        System.out.println("Споживач купив 1 товар");  
        System.out.println("Товарів на складі: " + product);  
        notify(); // пробуджує потік  
        return product;  
    }  
    synchronized void put() {  
        while (product >= 3) { // Якщо сховище повне, потік повинен чекати  
            try { wait(); } catch (InterruptedException e) {} // блокує виконання потоку, з якого він викликаний  
        }  
        product++;  
        System.out.println("Виробник додав 1 товар");  
        System.out.println("Товарів на складі: " + product);  
        notify(); // пробуджує потік  
    }  
}
```

Взаємодія потоків на прикладі

В методі **main()** головний потік запускає два дочірніх і відразу припиняється:

```
public static void main(String[] args) {  
    Store store = new Store();  
    Producer producer = new Producer(store);  
    Consumer consumer = new Consumer(store);  
    new Thread(producer).start();  
    new Thread(consumer).start();  
}
```

Виробник додав 1 товар
Товарів на складі: 1
Виробник додав 1 товар
Товарів на складі: 2
Виробник додав 1 товар
Товарів на складі: 3
Споживач купив 1 товар
Товарів на складі: 2
Споживач купив 1 товар
Товарів на складі: 1
Споживач купив 1 товар
Товарів на складі: 0
Виробник додав 1 товар
Товарів на складі: 1
Виробник додав 1 товар
Товарів на складі: 2
Споживач купив 1 товар
Товарів на складі: 1
Споживач купив 1 товар
Товарів на складі: 0

Взаємне блокування потоків (deadlock)

Пояснення:

Взаємне блокування (deadlock) виникає, коли два або більше потоків чекають один на одного, і через це ніхто не може продовжити виконання.

Приклад найпростішого випадку:

1. Потік 1 захоплює монітор **об'єкта А**.
2. Потік 2 захоплює монітор **об'єкта В**.
3. Потік 1 намагається захопити монітор **В**, але блокується, бо він вже зайнятий потоком 2.
4. Потік 2 намагається захопити монітор **А**, але теж блокується.

В результаті **обидва потоки чекають один на одного і не можуть продовжити роботу** — це і є deadlock.

Приклад взаємного блокування потоків

```
public class DeadLocker extends Thread{
    Object a, b;

    public DeadLocker(Object a, Object b, String name) {
        super(name);
        this.a = a;
        this.b = b;
    }

    public void run() {
        // введення монітору a
        synchronized(a) {
            System.out.println(getName() + ": step A");
            // передача процесору іншому потоку
            sleep(0);
            // введення монітору b
            synchronized(b) {
                System.out.println(getName() + ": step B");
            };
        }
    }
}
```

Приклад взаємного блокування потоків

Для демонстрації взаємного блокування потоків використовуємо наступний код:

```
// Об'єкти a та b, що слугують моніторами  
Object a = new Object(),  
Object b = new Object();  
  
// Перший потік отримує об'єкти a та b  
new DeadLocker(a,b,"First").start();  
  
// Другий потік отримує ті ж об'єкти в іншому порядку  
new DeadLocker(b,a,"Second").start();
```

Цей код виведе:

First: step A

Second: step A

після чого потоки заблокуються.

Якщо виклик методу `sleep(0)` замінити на виклик методу `a.wait()`, та додати `b.notify()`, взаємного блокування потоків не відбудеться.

Переривання потоків

Механізм переривання в класі Thread підтримується методами:

- **interrupt()** – встановлює стан потоку в значення “перерваний”;
- **isInterrupted()** – перевіряє чи перерваний потік;
- **interrupted()** – те ж саме, але ще і скидає його.

Взаємодія потоків на прикладі

```
class JThread extends Thread {  
    JThread(String name) {  
        super(name);  
    }  
    public void run() {  
        System.out.printf("%s started... \n", Thread.currentThread().getName());  
        int counter = 1; // лічильник циклів  
        while (!isInterrupted()) { // отримуємо статус поточного потоку, якщо поверне true то ми вийдемо з циклу  
            System.out.println("Loop " + counter++);  
        }  
        System.out.printf("%s finished... \n", Thread.currentThread().getName());  
    }  
}
```

```
public class Program {  
    public static void main(String[] args) {  
        System.out.println("Main thread started...");  
        JThread t = new JThread("JThread");  
        t.start();  
        try{  
            Thread.sleep(150);  
            t.interrupt(); // Виклик цього методу встановлює потоку статус, що він перерваний.  
        } catch (InterruptedException e) { System.out.println("Thread has been interrupted"); }  
        System.out.println("Main thread finished...");  
    }  
}
```

```
Main thread started...  
JThread started...  
Loop 1  
Loop 2  
Loop 3  
Loop 4  
JThread finished...  
Main thread finished...
```

Lock – інтерфейс, що позначає м'ютекс у явному вигляді.

При цьому набагато гнучкіший, ніж стандартні java-монітори.

Основні відмінності від synchronized-блоків:

- Ви самі створюєте об'єкт-м'ютекс;
- Ви самі вирішуєте які ресурси захищати;
- Ви самі відповідальні за звільнення м'ютексу;
- Можна захоплювати м'ютекс в одному контексті, а відпускати в іншому.

Реалізація синхронізації на **Lock**'ах у багатьох випадках ефективніше за використання synchronized-блоків.

Через велику гнучкість вона дозволяє накладити більш слабкі умови на взаємодію потоків.

Lock-и важче відлагоджувати і діагностувати проблеми, адже з точки зору JVM це рядові об'єкти. Для synchronized-блокувань завжди можна запросити у JVM Thread dump, що покаже всі потоки та взяті ними synchronized-блокування.

Розглянемо реалізацію thread-safe лічильника з використанням Lock.

На відміну від synchronized Lock не є засобом мови, а є звичайним об'єктом з набором методів.

В цьому випадку критичну секцію обмежують операції **lock()** та **unlock()**.

```
public class Counter{  
  
    private Lock lock = new ReentrantLock();  
    private int count = 0;  
  
    public int increment(){  
        lock.lock();  
        int newCount = ++count;  
        lock.unlock();  
        return newCount;  
    }  
}
```

Виклик **lock()** блокує, якщо **Lock** в даний момент зайнятий, тому зручно використовувати метод **tryLock()**, який відразу поверне управління та результат.

При використанні **Lock** не будуть працювати стандартні методи **wait()**, **notify()** та **notifyAll()**, адже монітор як такий не використовується.

Замість них використовуються реалізації інтерфейсу **Condition**, асоційовані з **Lock**: для цього необхідно викликати **Lock.newCondition()** і вже у **Condition** викликати методи **await()**, **signal()** та **signalAll()**.

З одним **Lock** можна асоціювати декілька **Condition**.

Найбільш розповсюджений патерн для роботи з **Lock**'ами представлений праворуч.

```
lock.lock();  
try {  
    // критична секція  
} finally {  
    lock.unlock(); // гарантоване звільнення  
}
```

Він гарантує, що **Lock** буде відпущений в будь-якому випадку, навіть якщо при роботі з ресурсом буде викидатися виключення.

Для `synchronized` цей підхід неактуальний – там засобами мови надається гарантія, що м'ютекс буде відпущений.

Цей патерн досить корисний для будь-якої ситуації, що вимагає обов'язкового вивільнення ресурсів.

Широко використовуються дві основні реалізації **Lock**:

- **ReentrantLock** допускає вкладені критичні секції;
- **ReadWriteLock** має різні механізми блокування на читання та запис, що дозволяє зменшити накладні ресурси.

Callable

Має єдиний метод **V call()**.

За принципом дії схожий з **Runnable**.

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```

Як бачимо, у нього так само, як і у **Runnable**, лише один метод. Призначення методу таке ж, як і у методу `run`, — він міститиме код, який буде виконуватись у паралельній нитці. З відмінностей це значення, що повертається **V**. Тепер це може бути будь-який тип, який ми вкажемо під час реалізації інтерфейсу.

Ще з корисностей: метод `call` може прокидувати **Exception**, так що на відміну від методу `run`, тут можна не стежити за винятками, що перевіряються, які виникають усередині методу

Executor

Executor – інтерфейс, що означає абстрактну систему для асинхронного виконання завдань.

У нього передають виконуваний код, а він турбується про вибір потоку виконання.

При цьому він може містити декілька потоків, забезпечуючи їх ефективне перевикористання.

Клас **Executors** являє собою фабрику для створення Executor'ів.

Ця фабрика дозволяє створювати різні черги та пули потоків, що звільняє програміста від необхідності писати одноманітний інфраструктурний код.

Простий приклад використання Executor'у представлений нижче.

```
ExecutorService executor = Executors.newSingleThreadExecutor();
    executor.execute(new Runnable() {
        @Override
        public void run() {
            // Цей код буде виконано в окремому потоці
        }
    });
```

ExecutorService

Являє собою розширення Executor'у з додатковими можливостями.

Здатний створювати об'єкти Future<T>, що являють собою результати виконання асинхронних операцій.

Основні методи:

- **submit(...)** – різні варіації цього методу приймають задачу до виконання;
- **invokeAll()** – метод виконає переданий у нього список задач і поверне управління тоді, коли всі задачі будуть завершені або наступить таймаут;
- **invokeAny()** – виконає переданий у нього список задач і поверне управління тоді, коли хоча б одна задача буде виконана або наступить таймаут;
- **shutdown()** – при виклику цього методу ExecutorService закінчить виконання поточних задач, але нових вже не прийматиме.

Багато Executor'ів, що повертаються фабрикою Executors насправді є реалізаціями ExecutorService.

ExecutorService

```
public class Main {  
    public static void main(String[] args) {  
        ExecutorService executor = Executors.newFixedThreadPool(10);  
        for (int i = 0; i < 10; i++) {  
            executor.submit(() -> {  
                System.out.println("Поточний потік: " + Thread.currentThread().getName());  
            });  
        }  
        executor.shutdown();  
    }  
}
```

У цьому прикладі `ExecutorService` створює пул з 10 потоків за допомогою методу `Executors.newFixedThreadPool(10)`. Потім ми використовуємо цей пул для відправлення 10 асинхронних задач до виконання за допомогою методу `submit()`. Кожна задача виводить ім'я поточного потоку, яке можна отримати за допомогою `Thread.currentThread().getName()`. Нарешті, ми викликаємо `shutdown()` для закриття пула потоків після завершення всіх задач.

Future

Future – інтерфейс, що семантично позначає результат виконання асинхронної операції.

Тип-параметр – це тип результату операції.

Важливі методи інтерфейсу:

- **get()** дозволяє отримати результати операції, блокуючи, якщо результату ще немає;
- **cancel()** зупиняє виконання задачі, тільки якщо вона ще не завершена;
- **isDone()** дозволяє виявити чи завершена задача.

Усі операції в межах виконання задачі `happens-before` будь яких операцій після виклику методу **get()**.

Найрозповсюдженішою реалізацією **Future** є **FutureTask**.

FutureTask також реалізує **Runnable**, так що його екземпляри зручно передавати в **Thread** або **Executor**.

CompletableFuture

CompletableFuture - це розширення Future API.

Future використовується як посилання на результат асинхронного завдання. У ньому є метод `isDone()` для перевірки, чи завершилося завдання чи ні, а також метод `get()` для отримання результату після його завершення.

Можливе виконання асинхронної задачі та повернення результату з використанням `supplyAsync()`

Ви можете використовувати метод `thenApply()` для обробки та перетворення результату `CompletableFuture` при його надходженні. Як аргумент він приймає `Function<T, R>`.

Приклад з CompletableFuture

```
public class Main {  
    public static void main(String[] args) throws InterruptedException, ExecutionException {  
        // Створення CompletableFuture з асинхронним виконанням  
        CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> "Hello, world!");  
  
        // Додавання обробника для результату  
        future.thenAccept(result -> System.out.println("Результат: " + result));  
  
        // Очікування завершення асинхронної операції  
        future.get(); // Це блокуючий виклик, який очікує завершення майбутнього  
  
        // Затримка виконання головного потоку для виведення результату  
        Thread.sleep(1000);  
    }  
}
```

Створюємо `CompletableFuture`, який повертає рядок "Hello, world!" асинхронно за допомогою методу `supplyAsync()`. Потім ми додаємо обробник, який виводить результат у консоль за допомогою `thenAccept()`. Нарешті, ми очікуємо завершення асинхронної операції з використанням методу `get()`, який є блокуючим.



Рефлексія. Аннотації

Рефлексія

Рефлексія - це механізм, який дозволяє отримувати інформацію про класи, інтерфейси, поля та методи програми під час її виконання. Основна ідея рефлексії полягає в тому, щоб програма могла аналізувати та маніпулювати своєю структурою в час виконання. За допомогою рефлексії можна отримувати доступ до поля, викликати методи, створювати нові об'єкти та багато іншого.

Рефлексія дозволяє створювати більш гнучкі, універсальні та динамічні програми, а також використовується у різних бібліотеках та фреймворках для реалізації різноманітних функцій, таких як серіалізація, робота з анотаціями, створення об'єктів за рядком тощо.

Рефлексія (можливості)

1. **Створення об'єктів:** Рефлексія дозволяє створювати нові об'єкти класів у час виконання, навіть якщо вони не відомі на етапі компіляції. Це може бути корисним, наприклад, для створення об'єктів з параметрами, які визначаються у виконанні програми.
2. **Модифікація класів:** Рефлексія дозволяє змінювати поля, методи та інші характеристики класів у час виконання. Це може бути використано для динамічної модифікації поведінки програми або взаємодії з бібліотеками, які використовують анотації для конфігурації.
3. **Робота з анотаціями:** Рефлексія дозволяє отримувати доступ до анотацій, що застосовані до класів, полів, методів та інших елементів програми. Це дозволяє реалізувати різноманітні функції, такі як автоматична обробка коду або конфігурація програми на основі анотацій.
4. **Взаємодія з класами:** Рефлексія дозволяє отримувати доступ до інформації про класи, завантажені в систему, та взаємодіяти з ними. Це може бути корисно для реалізації власних загрузчиків класів або роботи з динамічно завантаженими модулями.

Основні методи класу `java.lang.Class`

1. **`getMethods()`** - повертає масив об'єктів `Method`, які представляють всі публічні методи цього класу та його суперкласів.
2. **`getDeclaredMethods()`** - повертає масив об'єктів `Method`, які представляють всі методи цього класу, незалежно від модифікаторів доступу.
3. **`getFields()`** - повертає масив об'єктів `Field`, які представляють всі публічні поля цього класу та його суперкласів.
4. **`getDeclaredFields()`** - повертає масив об'єктів `Field`, які представляють всі поля цього класу, незалежно від модифікаторів доступу.
5. **`getConstructors()`** - повертає масив об'єктів `Constructor`, які представляють всі публічні конструктори цього класу.
6. **`getDeclaredConstructors()`** - повертає масив об'єктів `Constructor`, які представляють всі конструктори цього класу, незалежно від модифікаторів доступу.
7. **`getField(String name)`** - повертає об'єкт `Field`, який представляє поле з вказаним ім'ям.
8. **`newInstance()`** - створює новий об'єкт цього класу.

Приклад 1

```
public class ReflectionExample {  
    public static void main(String[] args) throws Exception {  
        // Створюємо об'єкт класу "String"  
        String str = "Hello, world!";  
  
        // Отримуємо метод "toUpperCase" класу "String"  
        Method method = String.class.getMethod("toUpperCase");  
  
        // Викликаємо метод "toUpperCase" для об'єкта "str"  
        String result = (String) method.invoke(str);  
  
        // Виводимо результат  
        System.out.println(result);  
    }  
}
```

Результат:

HELLO, WORLD!

Приклад 2

```
public class ReflectionExample {  
    public static void main(String[] args) throws Exception {  
        // Створюємо об'єкт класу "String"  
        String str = "Hello, world!";  
  
        // Отримуємо поле "value" класу "String"  
        Field field = String.class.getDeclaredField("value");  
        field.setAccessible(true); // Дозволяємо доступ до приватного поля value класу String  
  
        // Отримуємо значення поля "value"  
        char[] value = (char[]) field.get(str);  
  
        // Виводимо значення поля "value"  
        System.out.println(value);  
    }  
}
```

Результат:

"Hello, world!"

Приклад 3

```
public class ReflectionExample {  
    public static void main(String[] args) throws Exception {  
        // Отримуємо клас "String"  
        Class<?> stringClass = String.class;  
  
        // Виводимо ім'я класу  
        System.out.println("Class name: " + stringClass.getName());  
  
        // Отримуємо всі методи класу "String"  
        Method[] methods = stringClass.getMethods();  
  
        // Виводимо список методів  
        System.out.println("Methods:");  
  
        for (Method method : methods) {  
            System.out.println(method.getName());  
        }  
    }  
}
```

Результат ->

```
Class name: java.lang.String  
Methods:  
equals  
hashCode  
toString  
length  
isEmpty  
charAt  
codePointAt  
codePointBefore  
codePointCount  
offsetByCodePoints  
getChars  
getBytes  
compareTo  
compareToIgnoreCase  
equalsIgnoreCase  
regionMatches  
startsWith  
endsWith  
indexOf  
lastIndexOf  
indexOf  
lastIndexOf  
substring  
subSequence  
concat  
replace  
...
```

Аннотації

Аннотації - це спеціальні метадані, які можна додавати до класів, методів, змінних та інших елементів програми для надання додаткової інформації про їх функції та властивості.

Анотації

Ось кілька основних аспектів анотацій:

Декларація: Анотації декларуються за допомогою символу @ перед ім'ям анотації. Наприклад: @Override, @Deprecated, @SuppressWarnings.

Використання: Анотації можна використовувати на рівні класів, методів, полів, параметрів методів, локальних змінних та інших елементів програми.

Вбудовані анотації: Java має кілька вбудованих анотацій, таких як @Override, @Deprecated, @SuppressWarnings, які надають додаткову інформацію про код.

Власні анотації: Розробники можуть створювати свої власні анотації, використовуючи ключове слово **@interface**. Вони можуть мати свої власні елементи, які можна використовувати для передачі параметрів анотації.

Обробка анотацій: Анотації можуть бути оброблені за допомогою рефлексії, що дозволяє вам визначати анотації, вказані для класів, методів, полів тощо, та виконувати певні дії на основі цих анотацій.

Використання в інструментах:

Аннотації часто використовуються в інструментах розробки, таких як системи збірки проектів (наприклад, Maven або Gradle), фреймворки (наприклад, Spring або Hibernate) і засоби тестування (наприклад, JUnit), щоб визначати специфічні налаштування, вимоги або метадані.

Загалом, аннотації дозволяють додавати додаткову інформацію до програмного коду, що спрощує розробку, покращує читабельність коду і надає можливості для автоматичної обробки та аналізу програми за допомогою різноманітних інструментів.

Аннотація @Retention

Аннотація **@Retention** визначає рівень утримання (retention level) для анотації у Java. Утримання визначає, на якому етапі життєвого циклу програми анотація залишається доступною.

```
@Retention(RetentionPolicy.RUNTIME)  
@interface MyAnnotation {}
```

В Java існують три рівні утримання:

RetentionPolicy.SOURCE: Аннотація доступна лише під час компіляції і не зберігається в скомпільованих файлів.

RetentionPolicy.CLASS: Аннотація доступна під час компіляції та зберігається в скомпільованих файлів, але не доступна в рантаймі (під час виконання програми).

RetentionPolicy.RUNTIME: Аннотація доступна під час компіляції, зберігається в скомпільованих файлах і також доступна в рантаймі (під час виконання програми). Це найбільш потужний рівень утримання, оскільки дозволяє програмі звертатися до анотацій під час виконання.

Анотація @Target

Ще одна важлива анотація - це @Target. Ця анотація визначає, на яких елементах програми можна використовувати дану анотацію. Для прикладу:

```
@Target({ElementType.METHOD, ElementType.FIELD}) // Дозволяє використовувати анотацію на методах та полях
@interface MyAnnotation {
    // Параметри анотації
}
```

У Java є кілька типів елементів (ElementType), на яких можна застосовувати анотації. Ось деякі з них:

ElementType.TYPE: Застосовується до класів, інтерфейсів, або еnumерацій.

ElementType.FIELD: Застосовується до полів класу, включаючи enum-константи.

ElementType.METHOD: Застосовується до методів класу.

ElementType.PARAMETER: Застосовується до параметрів методу.

ElementType.CONSTRUCTOR: Застосовується до конструкторів класу.

ElementType.LOCAL_VARIABLE: Застосовується до локальних змінних.

ElementType.ANNOTATION_TYPE: Застосовується до анотацій.

ElementType.PACKAGE: Застосовується до пакетів.

ElementType.TYPE_PARAMETER: Застосовується до параметрів типу (введених у деклараціях параметрів типу методу, класу або інтерфейсу).

ElementType.TYPE_USE: Застосовується до використання типів.

Аннотації (приклад)

Аннотація для класу з вказанням назви та версії:

```
@Retention(RetentionPolicy.RUNTIME)
@interface ClassInfo {
    String name();
    double version();
}

@ClassInfo(name = "MyClass", version = 1.0)
class MyClass {

    public void display() {
        System.out.println("This is MyClass.");
    }
}
```

У цьому прикладі ми створили аннотацію `@ClassInfo`, яка має два поля: **name** та **version**. Потім ми застосували цю аннотацію до класу `MyClass`, вказавши його назву та версію.

Завдяки `@Retention(RUNTIME)` ми можемо зчитати ці дані через рефлексію. Наступний слайд

Анотації (приклад)

```
public class Main {  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj.display();  
  
        // Отримуємо інформацію про анотацію класу MyClass  
        Class<?> clazz = obj.getClass();  
        Annotation annotation = clazz.getAnnotation(ClassInfo.class);  
        if (annotation instanceof ClassInfo) {  
            ClassInfo classInfo = (ClassInfo) annotation;  
            System.out.println("Class Name: " + classInfo.name());  
            System.out.println("Class Version: " + classInfo.version());  
        }  
    }  
}
```

Результат:

This is MyClass.

Class Name: MyClass

Class Version: 1.0

Аннотації (приклад 2)

```
// Оголошення нової анотації
@interface MyAnnotation {
    String value();
}

// Використання анотації на рівні класу
@MyAnnotation(value = "Це моя перша анотація")
public class MyClass {
    // Код класу

    // Метод, який використовує анотацію
    @MyAnnotation(value = "Це мій перший метод")
    public void myMethod() {
        // Код методу
    }
}
```

Анотації (приклад 2)

```
public class MainClass {
    public static void main(String[] args) {
        // Дістаємо анотації класу за допомогою рефлексії
        Class<MyClass> obj = MyClass.class;
        Annotation[] classAnnotations = obj.getAnnotations();
        for (Annotation annotation : classAnnotations) {
            if (annotation instanceof MyAnnotation) {
                MyAnnotation myAnnotation = (MyAnnotation) annotation;
                System.out.println("Анотація класу: " + myAnnotation.value());
            }
        }

        // Дістаємо анотації методу за допомогою рефлексії
        try {
            Method method = obj.getMethod("myMethod");
            Annotation[] methodAnnotations = method.getAnnotations();
            for (Annotation annotation : methodAnnotations) {
                if (annotation instanceof MyAnnotation) {
                    MyAnnotation myAnnotation = (MyAnnotation) annotation;
                    System.out.println("Анотація методу: " + myAnnotation.value());
                }
            }
        } catch (NoSuchMethodException e) {
            e.printStackTrace();
        }
    }
}
```



Дякую за увагу!